Practical : 12	Implementation of Travelling Salesman Problem
Date : __ / __ / 2026	

Aim: To implement the **Travelling Salesman Problem (TSP)** in C++ using **Dynamic Programming (Held-Karp algorithm)** to find the minimum cost tour that visits every city exactly once and returns to the starting city.

Objective :- To represent the cities with a cost matrix, solve TSP using bitmask dynamic programming, print the minimum cost and the optimal tour, and to analyze the time and space complexity compared with the brute force method.

A. Problem Representation (Cost Matrix)

1. Start.
2. Read the number of cities n.
3. Read an n x n cost matrix, where cost[i][j] is the cost of travelling from city i to city j.
4. Display the cost matrix.
5. Stop.

B. TSP Algorithm (Dynamic Programming with Bitmask)

1. Start.
2. Represent a set of visited cities as a bitmask (bit i is 1 if city i is visited). Let dp[mask][i] be the minimum cost to start at city 0, visit exactly the cities in mask and end at city i.
3. Set all dp values to infinity and set dp[1][0] = 0 (only city 0 is visited).
4. For every mask that contains city 0, and every city i in mask with a finite dp[mask][i]:
5. For every city j not in mask, let newMask = mask | (1 << j). If dp[mask][i] + cost[i][j] < dp[newMask][j], update it and set parent[newMask][j] = i.
6. Let full = (1 << n) - 1. The minimum tour cost is the minimum, over all cities i other than 0, of dp[full][i] + cost[i][0].
7. Rebuild the tour by walking back through parent[][] from the best last city.
8. Print the minimum cost and the optimal tour.
9. Stop.

Program A: Travelling Salesman Problem (practical12_tsp.cpp)

```
#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;
class TSP {
    int n;
    vector<vector<int>> cost;
    vector<vector<int>> dp;        // dp[mask][i] = min cost to visit cities in mask, ending at i
    vector<vector<int>> parent;    // used to rebuild the tour
public:
    TSP(int cities) {
        n = cities;
        cost.assign(n, vector<int>(n, 0));
    }
    void setCost(int i, int j, int c) {
        cost[i][j] = c;
    }
    // Display the cost matrix
    void displayMatrix() {
        cout << "\n--- Cost Matrix ---" << endl;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cout << cost[i][j] << "\t";
            }
            cout << endl;
        }
    }
    // Travelling Salesman Problem using Dynamic Programming (Held-Karp)
    void solve() {
        int FULL = (1 << n);
        dp.assign(FULL, vector<int>(n, INT_MAX));
        parent.assign(FULL, vector<int>(n, -1));
        dp[1][0] = 0;                  // start at city 0, only city 0 visited
        for (int mask = 1; mask < FULL; mask++) {
            if (!(mask & 1)) continue;         // every tour must contain city 0
            for (int i = 0; i < n; i++) {
                if (!(mask & (1 << i)) || dp[mask][i] == INT_MAX) continue;
                for (int j = 0; j < n; j++) {
                    if (mask & (1 << j)) continue;     // j already visited
                    int newMask = mask | (1 << j);
                    int newCost = dp[mask][i] + cost[i][j];
                    if (newCost < dp[newMask][j]) {
                        dp[newMask][j] = newCost;
                        parent[newMask][j] = i;
                    }
                }
            }
        }
        // Close the tour: return from the last city to city 0
        int best = INT_MAX, last = -1;
        for (int i = 1; i < n; i++) {
            if (dp[FULL - 1][i] != INT_MAX &&
                dp[FULL - 1][i] + cost[i][0] < best) {
                best = dp[FULL - 1][i] + cost[i][0];
                last = i;
            }
        }
        // Rebuild the path by walking back through parent[][]
        vector<int> path;
        int mask = FULL - 1, cur = last;
        while (cur != -1) {
            path.push_back(cur);
            int p = parent[mask][cur];
            mask ^= (1 << cur);
            cur = p;
        }
        reverse(path.begin(), path.end());
```

```
        cout << "\n--- TSP Result ---" << endl;
        cout << "Minimum tour cost = " << best << endl;
        cout << "Optimal tour      : ";
        for (int city : path) {
            cout << city << " -> ";
        }
        cout << 0 << endl;
    }
};
int main() {
    int n;
    cout << "Enter number of cities: ";
    cin >> n;
    if (n < 2 || n > 15) {
        cout << "Number of cities must be between 2 and 15." << endl;
        return 0;
    }
    TSP t(n);
    cout << "Enter cost matrix (" << n << " x " << n << "):" << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int c;
            cin >> c;
            t.setCost(i, j, c);
        }
    }
    t.displayMatrix();
    t.solve();
    return 0;
}
```

Output:

Sample used: 4 cities (0 to 3) with the cost matrix shown below. The minimum tour cost is 80 for the tour 0, 2, 3, 1, 0 (15 + 30 + 25 + 10). The reverse tour 0, 1, 3, 2, 0 has the same cost.

```
Output
Enter number of cities: 4
Enter cost matrix (4 x 4):
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

--- Cost Matrix ---
0       10      15      20
10      0       35      25
15      35      0       30
20      25      30      0

--- TSP Result ---
Minimum tour cost = 80
Optimal tour      : 0 -> 2 -> 3 -> 1 -> 0


=== Code Execution Successful ===
```

Program B: Performance Analysis (practical12_analysis.cpp)

```
#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>
#include <random>
#include <chrono>
using namespace std;
using namespace chrono;
// Held-Karp dynamic programming; returns minimum tour cost
int tspDP(const vector<vector<int>> &cost, int n) {
    int FULL = 1 << n;
    vector<vector<int>> dp(FULL, vector<int>(n, INT_MAX));
    dp[1][0] = 0;
    for (int mask = 1; mask < FULL; mask++) {
        if (!(mask & 1)) continue;
        for (int i = 0; i < n; i++) {
            if (!(mask & (1 << i)) || dp[mask][i] == INT_MAX) continue;
            for (int j = 0; j < n; j++) {
                if (mask & (1 << j)) continue;
                int newMask = mask | (1 << j);
                dp[newMask][j] = min(dp[newMask][j], dp[mask][i] + cost[i][j]);
            }
        }
    }
    int best = INT_MAX;
    for (int i = 1; i < n; i++)
        best = min(best, dp[FULL - 1][i] + cost[i][0]);
    return best;
}
// Brute force: try every permutation of cities 1 .. n-1
int tspBruteForce(const vector<vector<int>> &cost, int n) {
    vector<int> perm;
    for (int i = 1; i < n; i++) perm.push_back(i);
    int best = INT_MAX;
    do {
        int total = cost[0][perm[0]];
        for (int i = 0; i + 1 < n - 1; i++)
            total += cost[perm[i]][perm[i + 1]];
        total += cost[perm[n - 2]][0];
        best = min(best, total);
    } while (next_permutation(perm.begin(), perm.end()));
    return best;
}
// Build a random symmetric cost matrix (fixed seed)
vector<vector<int>> generateCost(int n) {
    mt19937 rng(42);
    vector<vector<int>> cost(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            cost[i][j] = cost[j][i] = rng() % 100 + 1;
    return cost;
}
int main() {
    int sizes[] = {5, 8, 10, 11, 12, 14, 15};
    cout << "TSP Algorithm Performance Analysis" << endl << endl;
    for (int n : sizes) {
        vector<vector<int>> cost = generateCost(n);
        int dpCost = 0;
        long long dpTime = LLONG_MAX;
        for (int run = 0; run < 3; run++) {          // best of 3 runs
            auto s1 = high_resolution_clock::now();
            dpCost = tspDP(cost, n);
            auto e1 = high_resolution_clock::now();
            dpTime = min(dpTime, (long long)duration_cast<microseconds>(e1 - s1).count());
        }
        cout << "Number of Cities   = " << n << endl;
        cout << "Held-Karp Cost     : " << dpCost << endl;
        cout << "Held-Karp Time     : " << dpTime << " microseconds" << endl;
```

```
        if (n <= 11) {                      // brute force is too slow beyond this
            int bfCost = 0;
            long long bfTime = LLONG_MAX;
            for (int run = 0; run < 3; run++) {      // best of 3 runs
                auto s2 = high_resolution_clock::now();
                bfCost = tspBruteForce(cost, n);
                auto e2 = high_resolution_clock::now();
                bfTime = min(bfTime, (long long)duration_cast<microseconds>(e2 - s2).count());
            }
            cout << "Brute Force Cost   : " << bfCost << endl;
            cout << "Brute Force Time   : " << bfTime << " microseconds" << endl;
        } else {
            cout << "Brute Force Cost   : Skipped (too slow)" << endl;
            cout << "Brute Force Time   : Skipped (too slow)" << endl;
        }
        cout << "---------------------------------------" << endl;
    }
    return 0;
}
```

Output:

Random symmetric cost matrices (costs 1 to 100) were used for n = 5, 8, 10, 11, 12, 14 and 15 cities. Brute force (all permutations) was run only up to n = 11 because it becomes too slow. Each time is the best of 3 runs.

```
TSP Algorithm Performance Analysis

Number of Cities   = 5
Held-Karp Cost     : 124
Held-Karp Time     : 4 microseconds
Brute Force Cost   : 124
Brute Force Time   : 2 microseconds
---------------------------------------
Number of Cities   = 8
Held-Karp Cost     : 160
Held-Karp Time     : 65 microseconds
Brute Force Cost   : 160
Brute Force Time   : 607 microseconds
---------------------------------------
Number of Cities   = 10
Held-Karp Cost     : 171
Held-Karp Time     : 400 microseconds
Brute Force Cost   : 171
Brute Force Time   : 46174 microseconds
---------------------------------------
Number of Cities   = 11
Held-Karp Cost     : 177
Held-Karp Time     : 830 microseconds
Brute Force Cost   : 177
Brute Force Time   : 475834 microseconds
---------------------------------------
Number of Cities   = 12
Held-Karp Cost     : 247
Held-Karp Time     : 3084 microseconds
Brute Force Cost   : Skipped (too slow)
Brute Force Time   : Skipped (too slow)
---------------------------------------
Number of Cities   = 14
Held-Karp Cost     : 211
Held-Karp Time     : 15377 microseconds
Brute Force Cost   : Skipped (too slow)
Brute Force Time   : Skipped (too slow)
---------------------------------------
Number of Cities   = 15
Held-Karp Cost     : 230
Held-Karp Time     : 34330 microseconds
Brute Force Cost   : Skipped (too slow)
Brute Force Time   : Skipped (too slow)
---------------------------------------
```

Observation:

Whenever both methods were run (n = 5, 8, 10 and 11) they found exactly the same minimum cost, which confirms that the dynamic programming solution is correct.

Adding one city (n = 10 to n = 11) made brute force about 10 times slower (46174 to 475834 microseconds), as expected from (n - 1)!, while Held-Karp became only about 2 times slower (400 to 830 microseconds), as

expected from $O(n^2 2^n)$. At n = 11 Held-Karp was already about 573 times faster, and it solved n = 15 in 34330 microseconds.

The exact timings depend on the computer and compiler, so a different machine will show different numbers, but the growth pattern stays the same.

Analysis:

Time Complexity Analysis

Let n be the number of cities.

Held-Karp Dynamic Programming:

There are 2^n subsets (masks) of cities. For each mask, the program tries every ending city i (n choices) and every next city j (n choices).

Time Complexity = $O(n^2 2^n)$

Note: the brute force method checks all (n - 1)! tours. For n = 15 this is about 8.7 x 10^{10} tours, while the DP needs only about 7.4 x 10^6 steps.

Space Complexity Analysis

The tables dp[][] and parent[][] each have 2^n x n entries.

Space Complexity = $O(n 2^n)$

The program limits n to 15 cities so that these tables stay small.

Comparison of Methods for TSP

Feature	Brute force	Dynamic programming (Held-Karp)	Nearest neighbour (greedy)
Result	Exact (optimal)	Exact (optimal)	Approximate, not always optimal
Time complexity	O(n!)	$O(n^2 2^n)$	$O(n^2)$
Space complexity	O(n)	$O(n 2^n)$	O(n)
Practical size	About 10 to 12 cities	About 20 to 25 cities	Very large inputs

Conclusion :-

The Travelling Salesman Problem is NP-hard. The Held-Karp dynamic programming method solves it exactly by reusing sub-problems, where each sub-problem is defined by the set of visited cities and the last city:

1. The program uses a **bitmask** to represent visited cities and reduces the time from O(n!) of brute force to **$O(n^2 2^n)$**, with **$O(n 2^n)$** space.

2. For the sample with 4 cities it finds the minimum tour cost **80** and prints the optimal tour 0 -> 2 -> 3 -> 1 -> 0.

3. The running time is still exponential, so this method is suitable only for a small number of cities; for larger inputs, heuristic or approximation algorithms are used.